

RAG Copilot Week 2 专属执行手册

只服务 enterprise-rag-copilot 项目

RAG 工程化 + 质量评估 + 性能模块

文档用途	把 Week2 执行手册中 RAG 项目相关内容单独提取，并重写成 Chenning 与 Codex 都能直接执行的每日任务书。
最高约束	完全按照《Vibe Coding 项目实践公约》：AI 输出是候选稿；主线任务先定接口；生产区必须工程化；性能和评估指标必须可复现。
本周目标	把 Week1 naive RAG v0 升级为 SQL/结构化工具 + 文档检索的混合问答工程雏形，具备 evidence、fallback、evaluation、performance tracing、cache demo 和基础压测。
版本日期	2026-06-10

目录 / 使用方式

- 0. 文档定位与 Week2 成功标准
- 1. 实践公约在 RAG Week2 的落地规则
- 2. Week2 架构升级范围与目录结构
- 3. 接口契约、数据契约与拒绝合并标准
- 4. Day1-Day7 每日执行任务卡
- 5. 本周最终验收清单
- 6. Codex Prompt 模板、Review 模板、失败记录模板、面试表达模板

使用建议：每天只把当天任务卡发给 Codex，不要一次性让 Codex 改完整个项目。每个任务先让 Codex 复述设计边界，再生成候选代码，最后由你按 Review Gate 检查。

0. 文档定位与 Week2 成功标准

本文件不是新的 4 周计划，也不是普通学习笔记，而是 RAG Copilot 项目 Week2 的单独执行 PDF。原 Week2 手册中的 RepoPilot、LeetCode、泛理论学习内容不作为本文主线；本文只保留与 enterprise-rag-copilot 直接相关的工程任务、理论问题和面试沉淀。

Week2 的核心变化：Week1 已经跑通 naive RAG v0，Week2 要进入“生产区雏形”。这意味着项目不能只证明“能回答”，而要证明“回答为什么可信、失败时如何处理、质量如何评估、性能如何记录”。

方向	Week2 必须完成的 RAG 结果
Hybrid RAG	支持 structured tools + document retrieval 的混合流程。订单、退款、商品信息走结构化工具；政策、物流、保修说明走文档检索；混合问题可以融合两类证据。
Intent Router	至少识别 order_status、logistics、refund、return_policy、warranty、hybrid、unknown。
Domain Adapter	电商 schema、repository、tools、metadata rules 只放在 domains/ecommerce/，不得污染 pipeline/。
Evidence Citation	回答必须返回 evidence list，且能同时表示 structured evidence 与 document evidence。
Fallback	低分检索、缺少订单号、未知意图、高风险/证据不足问题必须返回 fallback_reason。
Evaluation	至少 30 条 ecommerce_cases，覆盖结构化问题、文档问题、混合问题、失败问题，并能运行基础评估脚本。
Performance	记录 intent、tool/sql、retrieval、rerank/mock、LLM/mock、total latency；性能数字只能来自本地脚本或日志。
Cache / Load Test	实现 mock cache 或 Redis cache demo，并提供最小压测脚本与 docs/performance.md。
文档资产	更新 README，完成 docs/evaluation.md、docs/failure_cases.md、docs/performance.md、docs/api.md、docs/week2_summary.md。

1. 实践公约在 RAG Week2 的落地规则

Week2 的 RAG 项目已经不是探索区玩具代码，而是会进入 GitHub、简历和面试追问的生产区雏形。以下规则高于“快点跑通”的目标。

规则	Codex 与开发者都必须遵守
AI 输出是候选稿	Codex 可以生成代码，但不能自动合并。每个任务必须经过 Correctness、Readability、Maintainability、Evolvability 四项检查。
接口先行	intent router、domain tools、evidence builder、fallback handler、performance tracer 先定义输入输出和失败语义，再实

规则	Codex 与开发者都必须遵守
	现。
机制与策略分离	pipeline/ 放通用机制；domains/ecommerce/ 放业务策略。禁止把退款规则、订单状态规则写进 core retriever 或 controller。
证据优先	关键业务回答没有 evidence 不允许强答。answer_generator 不能凭模型补全缺失证据。
失败可解释	系统不知道时必须 fallback，并记录 fallback_reason，而不是输出模糊或编造答案。
评估集中管理	evaluation cases、metrics、eval runner 集中在 evaluation/，不能散落在业务代码中。
指标可复现	P95/P99、QPS、cache hit rate、latency 不允许凭感觉写，只能来自脚本、trace 或日志。
每天小步提交	每天 1-2 个可审查 commit，避免让 Codex 一次性改十几个文件。

2. Week2 架构升级范围与目录结构

Week2 不要求强行重建 Week1 目录。如果你 Week1 目录已经不同，保持原结构也可以，但必须补齐职责边界：pipeline/ 是通用 RAG 流程，domains/ecommerce/ 是电商领域适配，evaluation/ 是评估资产，scripts/ 是压测和工具脚本，docs/ 是工程留痕。

```

enterprise-rag-copilot/
app/
  main.py
api/
  routes.py
core/
  config.py
  logging_config.py
  errors.py
schemas/
  chat.py
  evidence.py
  trace.py
  evaluation.py
pipeline/
  intent_router.py
  document_loader.py
  chunker.py
  embedder.py
  vector_store.py
  retriever.py
  reranker.py
  evidence_builder.py
  fallback_handler.py
  prompt_builder.py
  answer_generator.py
  performance_tracer.py
  cache.py
domains/
  ecommerce/
    schema.py
    repository.py

```

```
tools.py
metadata_rules.py
prompts.py
adapter.py
evaluation/
  ecommerce_cases.json
  run_eval.py
  metrics.py
scripts/
  build_index.py
  load_test.py
docs/
  architecture.md
  api.md
  evaluation.md
  failure_cases.md
  performance.md
  week2_upgrade_plan.md
  week2_summary.md
data/ecommerce/
  docs/
    faq.md
    return_policy.md
    logistics_policy.md
    warranty_policy.md
  mock/
    orders.json
    products.json
    refunds.json
tests/
README.md
requirements.txt
.env.example
Dockerfile
```

3. 接口契约、数据契约与拒绝合并标准

3.1 ChatResponse 建议字段

```
ChatResponse:
  answer: str
  intent: str
  route: str          # structured_only / document_only / hybrid / fallback
  evidence: list[Evidence]
  fallback: bool
  fallback_reason: str | None
  trace_id: str
  trace: TraceInfo | None  # 可选: debug/local 模式返回
```

3.2 Evidence 建议字段

```
Evidence:
  evidence_id: str
  evidence_type: str          # structured / document
  source: str                 # orders / refunds / faq.md / return_policy.md
  content: str | dict
  score: float | None
  metadata: dict              # document_type, product_category, policy_version, scenario, etc.
```

3.3 ToolResult 建议字段

```
ToolResult:
  tool_name: str
  success: bool
  data: dict | list | None
  error: str | None
  latency_ms: float
  evidence_hint: dict | None
```

3.4 TraceInfo 建议字段

```
TraceInfo:
  trace_id: str
  intent: str
  route: str
  tool_latency_ms: float | None
  retrieval_latency_ms: float | None
  rerank_latency_ms: float | None
  llm_latency_ms: float | None
  total_latency_ms: float
  cache_hit: bool
  fallback_reason: str | None
```

3.5 拒绝合并标准

- 把电商退款、订单、保修规则直接写进 pipeline/ 的 PR，拒绝合并。
- 结构化问题只走向量检索，不调用 order/refund/product tool 的 PR，拒绝合并。
- 关键业务回答没有 evidence list 的 PR，拒绝合并。
- fallback 只返回一句“请稍后再试”，没有 fallback_reason 的 PR，拒绝合并。
- evaluation cases 写在测试函数里或业务代码里，没有集中 JSON/runner 的 PR，拒绝合并。
- docs/performance.md 出现未由脚本或日志支持的 P95/P99/QPS 数字，拒绝合并。
- Codex 一次性重写大量文件且你无法逐行 review 的 PR，拆分后再做。

4. Day1-Day7 每日执行任务卡

以下每一天都按照“目标 - 文件 - Codex 任务卡 - 实现步骤 - 验收命令 - Review Gate - commit - 失败记录 - 面试表达”组织。你每天可以直接复制当天 Codex 任务卡到 Codex，但要把当前仓库实际文件路径补充给它。

Day 1: Week1 v0 审阅 + Hybrid RAG 升级设计

当天目标

审阅 Week1 naive RAG v0，明确哪些问题必须从“单一路径 RAG”升级为“结构化工具 + 文档检索”的混合流程，并形成 Week2 升级计划。当天不急着重改代码，先定边界、

	接口和验收。
生产区边界	当天产出的接口、schema、router、tool、evaluation 或 tracer 都会被 Week 3/Week 4 继续依赖；Codex 只能生成候选稿，必须通过人工 Review Gate 后再合并。
不做事项	不一次性重写全项目；不把电商规则写进 core pipeline；不虚构指标；不为了演示效果绕过 fallback/evidence。

需要创建或修改的文件

- docs/week2_upgrade_plan.md：记录 Week1 v0 短板、Week2 升级目标、风险和日程。
- docs/api.md：补充 /chat 当前请求/响应契约，标注 Week2 需要新增的 evidence、fallback、trace 字段。
- tests/test_week1_baseline.py：固定 Week1 仍然应通过的 /health、/chat、evidence 基线。
- docs/failure_cases.md：创建空文档或加入 Day1 发现的 naive RAG 失败样例。
- README.md：新增 Week2 Roadmap 小节。

给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。Week1 已经有 naive RAG v0: /health、/chat、document loader、chunker、retriever、prompt、answer、evidence 雏形。

Goal:

不要立刻重写系统。请先审阅当前 v0，生成 Week2 升级计划，并补充最小 baseline tests，确保后续改动不会破坏 Week1 主路径。

Constraints:

1. 不新增复杂依赖。
2. 不把电商业务规则写进 pipeline/。
3. 不实现完整 Hybrid RAG，只做计划、契约和基线测试。
4. AI 输出只是候选稿，代码和文档要便于人工 review。

Interface & Data:

保持现有 /chat 可用。ChatResponse 后续将扩展 answer, intent, route, evidence, fallback, fallback_reason, trace_id。

Process:

先列出当前文件结构和 v0 链路，再写 docs/week2_upgrade_plan.md，最后补 tests/test_week1_baseline.py。

Review:

输出修改文件清单、风险点、验收命令和不应合并的情况。

建议实现步骤

1. 运行现有测试或手动 curl，确认 Week1 v0 仍然可用。
2. 写 docs/week2_upgrade_plan.md，至少包含：v0 当前链路、主要短板、Week2 目标模块、每日拆分、风险清单。
3. 把 /chat 契约写进 docs/api.md，先记录现状，再记录 Week2 目标响应结构。
4. 补 tests/test_week1_baseline.py，覆盖 /health、/chat 返回 answer、/chat 返回 evidence 三个基本点。
5. 在 docs/failure_cases.md 记录至少 2 个 v0 可能失败的例子：订单状态误走文档、政策问题检索为空或证据弱。

验收命令 / 手动验收

```
pytest tests/test_week1_baseline.py
curl -s http://localhost:8000/health
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "退货政策是什么?"}'
人工检查 docs/week2_upgrade_plan.md 是否能解释：为什么 Week2 要做 structured tools、intent router、evidence、fallback、evaluation、tracing。
```

人工 Review Gate

- upgrade plan 不是空泛口号，必须能映射到 Day2-Day7 的具体模块。
- baseline tests 不能依赖真实 LLM；应支持 mock LLM 或 keyword fallback。
- docs/api.md 必须写清字段语义，不只是贴 JSON。
- 没有出现虚构性能数字。

建议 commit

- docs: add week2 upgrade plan
- test: add week1 rag baseline checks

失败记录提示

- 记录 v0 目前最容易失败的 2-3 类问题。
- 记录今天是否发现某个模块职责边界不清，比如 retriever 是否承担了生成回答职责。

今日三句面试表达

- 我先审阅 Week1 naive RAG v0，而不是直接堆新功能，是为了固定基线，避免升级过程中破坏已有主路径。
- Week2 的升级重点是让系统从单一路径 RAG 变成 structured tools + document retrieval 的混合问答。
- 我会用 evidence、fallback、evaluation 和 trace 来证明项目不是“看起来能用”的 toy demo。

Day 2: Intent Router + Structured Tools

当天目标	实现电商 domain schema、repository、order/product/refund structured tools 和 intent router，让精确事实问题不再只靠向量检索。
生产区边界	当天产出的接口、schema、router、tool、evaluation 或 tracer 都会被 Week 3/Week 4 继续依赖；Codex 只能生成候选稿，必须通过人工 Review Gate 后再合并。
不做事项	不一次性重写全项目；不把电商规则写进 core pipeline；不虚构指标；不为了演示效果绕过 fallback/evidence。

需要创建或修改的文件

- domains/ecommerce/schema.py：定义 Order、Product、Refund 或对应 dataclass/Pydantic model。
- domains/ecommerce/repository.py：封装 mock JSON 或 SQLite 查询，禁止业务查询散落。
- domains/ecommerce/tools.py：实现 get_order_status、get_product_info、get_refund_status。
- pipeline/intent_router.py：实现至少 7 类 intent。
- schemas/chat.py：扩展 ChatResponse 的 intent、route、fallback、trace_id 字段。
- tests/test_ecommerce_tools.py：测试结构化工具。
- tests/test_intent_router.py：测试 intent 分类。

给 Codex 的任务卡

Context:

你正在 Week2 RAG 工程化阶段。目标是把精确业务事实从文档检索中拆出来，通过 domains/ecommerce/ 的 structured tools 查询。

Goal:

实现 ecommerce schema/repository/tools 和 intent_router。至少支持 order_status, logistics, refund, return_policy, warranty, hybrid, unknown。

Constraints:

1. 电商业务规则只能放在 domains/ecommerce/。

2. pipeline/intent_router.py 只能决定路由，不直接查询订单或退款。

3. structured tools 要返回结构化 ToolResult 或可转 evidence 的 dict。

4. 不能依赖真实数据库服务；可用 JSON repository 或 sqlite3 本地文件。

Interface & Data:

输入 user query, intent_router 返回 intent + route + required_slots。

输入 order_id/product_id/refund_id, domain tool 返回 structured result。

Process:

先定义 schema 和 ToolResult 格式, 再实现 repository, 最后实现 intent_router 和 tests。

Review:

输出每个 intent 的识别样例、structured tool 的失败路径和测试命令。

建议实现步骤

6. 先检查 data/ecommerce/mock/orders.json、products.json 是否存在; 补 refunds.json。
7. 定义 schema.py, 不追求复杂 ORM; 目标是字段清楚、可被 tool 查询。
8. 实现 repository.py, 把 JSON 读取和查询集中起来。不要让 API route 直接读 JSON。
9. 实现 tools.py: 订单状态、商品信息、退款状态至少各一个函数。
10. 实现 intent_router.py: 先用规则/关键词版本, 不急着用 LLM 分类。
11. 让 /chat 根据 intent 决定 route: structured_only、document_only、hybrid、fallback。
12. 补测试: 订单状态必须命中 order_status, 退款问题必须命中 refund, 无法识别的问题进入 unknown。

验收命令 / 手动验收

```
pytest tests/test_ecommerce_tools.py tests/test_intent_router.py
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "我的订单 EC1001 现在是什么状态?"}'
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "退款 RF1001 处理到哪一步了?"}'
```

人工检查: 订单状态回答是否来自 structured evidence, 而不是 faq.md。

人工 Review Gate

- intent_router 不允许同时承担 repository 查询职责。
- 工具查询失败必须有结构化错误, 如 order_not_found、missing_order_id。
- structured evidence 必须能进入 evidence_builder, 而不是只拼进 answer 字符串。
- hybrid 问题不必完美, 但必须预留路由。

建议 commit

- feat: add ecommerce structured tools
- feat: add intent router for hybrid rag

失败记录提示

- 记录 intent 错分案例, 例如“我的耳机能不能退, 订单 EC1001”可能同时包含 order 与 return_policy。
- 记录 structured tool 查询失败时的错误语义是否清楚。

今日三句面试表达

- 订单状态、退款状态和商品信息属于精确事实, 我用 structured tools 处理, 而不是让向量检索猜。
- intent router 的作用是把不同类型问题送到不同路径, 避免所有请求都走同一个 naive RAG pipeline。
- 我把电商查询逻辑放在 domain adapter 中, 保持 core pipeline 可迁移。

Day 3: Policy Docs + Metadata Filtering

当天目标

扩充政策文档, 并让文档 chunk 带 metadata。retriever 支持按 document_type、product_category、policy_version、applicable_scenario 过滤, 减少纯向量检索的误召回。

生产区边界

当天产出的接口、schema、router、tool、evaluation 或 tracer 都会被 Week 3/Week 4 继续依赖; Codex 只能生成

不做事项

不一次性重写全项目；不把电商规则写进 core pipeline；不虚构指标；不为了演示效果绕过 fallback/evidence。

需要创建或修改的文件

- data/ecommerce/docs/faq.md、return_policy.md、logistics_policy.md、warranty_policy.md：补充规则和样例。
- domains/ecommerce/metadata_rules.py：定义文档类型、产品类别、政策版本、适用场景规则。
- pipeline/document_loader.py：加载文档时附带文档级 metadata。
- pipeline/chunker.py：chunk 继承并补充 metadata。
- pipeline/retriever.py：支持 metadata_filter 参数或预留接口。
- tests/test_metadata_rules.py、tests/test_metadata_filtering.py。

给 Codex 的任务卡**Context:**

Week2 RAG 项目需要从 naive retrieval 升级到 metadata-aware retrieval。当前文档可能只有纯文本 chunk。

Goal:

为 FAQ/退货/物流/保修文档添加 metadata rules，并让 retriever 支持 metadata_filter。政策类问题应优先检索对应文档类型。

Constraints:

1. 不引入复杂外部向量数据库依赖；先在现有 vector/keyword store 上加 metadata filter。
2. metadata 规则属于 domains/ecommerce/，通用 retriever 只消费 metadata_filter。
3. 不把 product category 或 policy version 写死在 retriever。

Interface & Data:

Chunk.metadata 至少包含 document_type, product_category, policy_version, applicable_scenario, source。

retrieve(query, top_k=3, metadata_filter=None) 返回带 score 和 metadata 的 chunks。

Process:

先定义 metadata schema，再改 loader/chunker，最后改 retriever filtering 和 tests。

Review:

说明 metadata filtering 如何减少误召回，以及哪些规则仍是临时实现。

建议实现步骤

13. 把 policy docs 扩充到能覆盖 return_policy、logistics_policy、warranty_policy 三类场景。
14. 在 metadata_rules.py 中集中定义文档类型和 scenario 映射。
15. document_loader 输出 Document(content, source, metadata)。
16. chunker 输出 Chunk(content, source, metadata, chunk_id)。
17. retriever 增加 metadata_filter 参数：当 intent 是 warranty 时，优先 filter document_type=warranty_policy。
18. 补测试：退货问题不应优先召回 warranty 文档；保修问题不应优先召回 logistics 文档。

验收命令 / 手动验收

```
pytest tests/test_metadata_rules.py tests/test_metadata_filtering.py
```

```
python scripts/build_index.py
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query":"耳机保修多久?"}'
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query":"退货需要满足什么条件?"}'
```

人工检查 evidence.metadata.document_type 是否合理。

人工 Review Gate

- metadata rules 集中在 domain adapter 中, retriever 不知道电商业务细节。
- metadata_filter 不应导致无结果时直接崩溃; 应该允许 fallback 或放宽检索。
- policy docs 的版本号和适用场景要写明, 不要只写散文式 FAQ。
- 测试要覆盖至少一个误召回场景。

建议 commit

- feat: add ecommerce metadata rules
- feat: support metadata-aware retrieval

失败记录提示

- 记录 metadata 过滤过严导致召回为空的例子。
- 记录某个问题同时适用多个 document_type 时如何处理。

今日三句面试表达

- metadata filtering 让检索不只依赖语义相似度, 还能利用文档类型、产品类别和政策版本进行约束。
- 我把 metadata rules 放在 ecommerce domain adapter 中, 避免 core retriever 写死业务场景。
- 这一步提升了系统可迁移性, 因为换成 HR/IT domain 时只需要替换 metadata rules, 而不是重写 retriever。

Day 4: Evidence Builder + Citation 约束

当天目标	统一 structured evidence 与 document evidence, 让 /chat 返回可追溯证据, 并在 prompt/answer 生成中加入 “不得无证据强答” 的约束。
生产区边界	当天产出的接口、schema、router、tool、evaluation 或 tracer 都会被 Week 3/Week 4 继续依赖; Codex 只能生成候选稿, 必须通过人工 Review Gate 后再合并。
不做事项	不一次性重写全项目; 不把电商规则写进 core pipeline; 不虚构指标; 不为了演示效果绕过 fallback/evidence。

需要创建或修改的文件

- schemas/evidence.py: 完善 Evidence 字段。
- pipeline/evidence_builder.py: 统一 structured tool result 与 retrieved chunks。
- pipeline/prompt_builder.py: 把 evidence 注入 prompt, 并加入引用约束。
- pipeline/answer_generator.py: 保证 mock/真实 LLM 都遵守 evidence-first。
- tests/test_evidence_builder.py。
- docs/api.md: 更新 evidence 返回格式。

给 Codex 的任务卡

Context:

Week2 要让 RAG 回答可追溯, structured tool 和 document retriever 都应该输出可统一展示的 evidence。

Goal:

实现 evidence_builder, 并让 /chat 返回 answer + evidence list。prompt_builder 必须把 evidence 放进 prompt, answer_generator 不应编造 evidence 之外的信息。

Constraints:

1. retriever 只负责返回 chunks, 不生成最终回答。

2. answer_generator 不能凭空补充未在 evidence 中出现的业务事实。

3. evidence_builder 要能处理 structured evidence 和 document evidence。

4. 没有 evidence 的关键业务问题应进入 fallback。

Interface & Data:

build_evidence(tool_results=None, retrieved_chunks=None) -> list[Evidence]

```
build_prompt(query, evidence) -> prompt
```

Process:

先定义 Evidence schema, 再实现 builder, 再接入 /chat pipeline, 最后补 tests 和 API 文档。

Review:

给出 structured/document/hybrid 三类 evidence 示例。

建议实现步骤

19. 完善 Evidence schema: evidence_type、source、content、score、metadata。
20. 实现 evidence_builder: 结构化工具结果转 structured evidence, retrieved chunks 转 document evidence。
21. 修改 prompt_builder: prompt 明确要求“只能基于 Evidence 回答; 证据不足时说明无法确定”。
22. 修改 /chat pipeline: answer 生成前先 build evidence, answer 后返回 evidence list。
23. 补测试: order_status 返回 structured evidence; return_policy 返回 document evidence; hybrid 返回两类 evidence。

验收命令 / 手动验收

```
pytest tests/test_evidence_builder.py
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "订单 EC1001 是否已经发货?"}'
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "这个商品可以七天无理由退货吗?"}'
```

人工检查: response.evidence 不为空; source 能定位到 orders/refunds/products 或具体 policy md。

人工 Review Gate

- Evidence.content 不应只是一段不可解析字符串, structured evidence 至少保留关键字段。
- answer 中出现的关键事实必须能在 evidence 中找到。
- 如果 retrieved_chunks 为空, 不能硬生成“根据政策可知”。
- API 文档必须同步更新 evidence 格式。

建议 commit

- feat: add evidence builder
- docs: document evidence response contract

失败记录提示

- 记录 answer 提到但 evidence 没有支撑的情况。
- 记录 structured evidence 与 document evidence 冲突时当前系统如何处理。

今日三句面试表达

- 我没有让模型直接输出最终答案, 而是先构建 evidence, 再让回答生成模块基于证据回答。
- Evidence builder 统一了数据库查询结果和文档检索结果, 为 citation correctness 和 failure analysis 打基础。
- 回答带 evidence 可以降低幻觉, 也方便后续做自动评估和人工审查。

Day 5: Fallback Handler + Failure Cases

当天目标	实现低置信度、缺少关键信息、未知意图、高风险/证据不足问题的 fallback handler, 并把失败案例沉淀到 docs/failure_cases.md 和 evaluation cases。
生产区边界	当天产出的接口、schema、router、tool、evaluation 或 tracer 都会被 Week 3/Week 4 继续依赖; Codex 只能生成候选稿, 必须通过人工 Review Gate 后再合并。
不做事项	不一次性重写全项目; 不把电商规则写进 core pipeline; 不虚构指标; 不为了演示效果绕过 fallback/evidence。

需要创建或修改的文件

- pipeline/fallback_handler.py: 集中处理 fallback 规则。
- core/errors.py: 定义错误码或 fallback reason 常量。
- schemas/chat.py: 确认 fallback、fallback_reason 字段。
- docs/failure_cases.md: 记录失败类型、触发输入、当前处理、后续改进。
- evaluation/ecommerce_cases.json: 加入失败类 cases。
- tests/test_fallback_handler.py。

给 Codex 的任务卡

Context:

Week2 RAG 项目不能在不知道时强答。fallback 是可靠性的核心，不是失败的遮羞布。

Goal:

实现 fallback_handler，处理 missing_order_id, order_not_found, low_retrieval_score, no_evidence, unknown_intent, high_risk_request 等场景。

Constraints:

1. fallback 逻辑必须集中，不允许散落在 routes.py、retriever.py、answer_generator.py。
2. fallback 必须返回 fallback_reason。
3. 低置信度或无证据时不能让 answer_generator 编造答案。
4. failure cases 要进入 docs/failure_cases.md 和 evaluation/ecommerce_cases.json。

Interface & Data:

should_fallback(intent, tool_results, retrieved_chunks, evidence, scores, required_slots) -> FallbackDecision

FallbackDecision: fallback: bool, reason: str, message: str, next_action: str | None

Process:

先定义 FallbackDecision，再实现规则，最后接入 /chat pipeline 和 tests。

Review:

列出每个 fallback_reason 的触发条件和用户可见回复。

建议实现步骤

24. 定义 fallback reason 常量: missing_order_id、order_not_found、low_retrieval_score、no_evidence、unknown_intent、high_risk_request。
25. 实现 fallback_handler.py: 输入 intent、evidence、retrieval score、tool result，输出 FallbackDecision。
26. 修改 /chat pipeline: 在 answer generation 前判断 fallback; fallback 时返回安全、明确、可操作的回答。
27. 把至少 8 条失败问题加入 evaluation/ecommerce_cases.json。
28. 更新 docs/failure_cases.md: 每类失败写触发样例、处理策略、当前局限。

验收命令 / 手动验收

```
pytest tests/test_fallback_handler.py
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "我的订单到哪里了?"}'
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "请帮我绕过退款审核"}'
```

```
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "一个完全不存在的问题 xyzabc"}'
```

人工检查: fallback=true 且 fallback_reason 明确，不出现强答。

人工 Review Gate

- fallback_handler 是唯一集中决策点，不允许多个模块各自写 fallback。
- fallback 回复不能太空泛，要告诉用户缺什么信息或下一步怎么做。

- 失败案例必须成为 evaluation cases，而不是只写在聊天记录里。
- 高风险问题不能因为检索到一些片段就强答。

建议 commit

- feat: add fallback handler
- docs: add rag failure cases

失败记录提示

- 记录 fallback 误触发：本来可以回答却被拦截。
- 记录 fallback 漏触发：证据不足却仍然回答。

今日三句面试表达

- Fallback 不是系统失败，而是可靠系统必须具备的拒答和转人工机制。
- 我把 fallback_reason 作为结构化字段返回，方便后续统计失败类型和优化系统。
- Week2 的 failure cases 会进入 evaluation，而不是只展示成功 demo。

Day 6: Evaluation Cases + Eval Runner + Performance Tracer

当天目标	建立基础评估闭环和性能追踪：至少 30 条 evaluation cases，能跑 eval runner；每次 /chat 记录 intent/tool/retrieval/rerank/LLM/total latency。
生产区边界	当天产出的接口、schema、router、tool、evaluation 或 tracer 都会被 Week 3/Week 4 继续依赖；Codex 只能生成候选稿，必须通过人工 Review Gate 后再合并。
不做事项	不一次性重写全项目；不把电商规则写进 core pipeline；不虚构指标；不为了演示效果绕过 fallback/evidence。

需要创建或修改的文件

- evaluation/ecommerce_cases.json：至少 30 条 case。
- evaluation/run_eval.py：运行 cases，调用本地 pipeline 或 API。
- evaluation/metrics.py：计算 intent accuracy、retrieval hit、fallback correctness、citation presence 等基础指标。
- pipeline/performance_tracer.py：记录阶段耗时。
- schemas/trace.py：定义 TraceInfo。
- docs/evaluation.md：说明评估集、指标、运行方式和局限。
- docs/performance.md：先写指标定义和采集方式，不虚构结果。
- tests/test_performance_tracer.py。

给 Codex 的任务卡

Context:

Week2 的 RAG 项目要证明效果不是“看起来还行”。本任务建立基础 evaluation 和 performance tracing。

Goal:

准备至少 30 条 evaluation cases，并实现 run_eval.py 与 performance_tracer.py。trace 需要记录 intent/tool/retrieval/rerank/LLM/total latency。

Constraints:

1. 不完整接入 RAGAS；先做自定义可解释评估。

2. 指标必须来自脚本输出，不能手写虚构。

3. evaluation cases 集中在 evaluation/，不要散落在 tests 或业务代码里。

4. tracer 不应强依赖真实 LLM；mock LLM 也要能记录 llm_latency_ms。

Interface & Data:

case fields: id, query, expected_intent, expected_route, required_evidence_keywords, expected_fallback, notes。

run_eval.py 输出 JSON/markdown summary。

Process:

先定义 case 格式，再写 30 条 cases，再写 runner 和 metrics，最后接入 tracer。

Review:

说明指标定义、样本覆盖范围和不能夸大的地方。

建议实现步骤

29. 设计 ecommerce_cases.json schema：结构化问题、文档问题、混合问题、失败问题各至少 6-8 条。
30. 实现 run_eval.py：逐条 case 调用 pipeline 或 /chat，收集 response。
31. 实现 metrics.py：intent accuracy、retrieval hit、fallback correctness、evidence present。
32. 实现 performance_tracer.py：用 context manager 或 start/end 记录阶段耗时。
33. 把 trace_id 写入 /chat response；debug 模式可返回 trace 摘要。
34. 写 docs/evaluation.md：评估目标、cases 分布、指标定义、运行命令、局限。
35. 写 docs/performance.md：指标定义、采集方式、测试环境占位。Day7 再补结果。

验收命令 / 手动验收

```
python -m evaluation.run_eval --cases evaluation/ecommerce_cases.json --out evaluation/eval_report.json
pytest tests/test_performance_tracer.py
curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query": "订单 EC1001 的退款状态和退货政策是什么?"}'
```

人工检查 evaluation/eval_report.json 是否包含每条 case 的 intent、route、fallback、evidence 命中信息。

人工检查 trace 是否包含 total_latency_ms。

人工 Review Gate

- eval runner 失败不能直接吞掉错误，要记录 case_id 和 error。
- evaluation metrics 只能说“本地 mock 数据集上的结果”，不能包装成生产指标。
- tracer 不应让业务代码变得混乱，最好用小型 helper/context manager。
- docs/evaluation.md 必须写局限：case 少、mock 数据、LLM/mock LLM 差异。

建议 commit

- feat: add rag evaluation runner
- feat: add performance tracer

失败记录提示

- 记录哪些 evaluation cases 失败，失败原因属于 intent、retrieval、evidence 还是 fallback。
- 记录 trace 显示的最慢阶段。

今日三句面试表达

- 我用 evaluation cases 和 runner 评估 RAG，而不是凭主观感觉判断回答质量。
- 我的指标包括 intent accuracy、retrieval hit、fallback correctness 和 evidence presence，后续可以扩展到 faithfulness。
- Performance tracer 让我能定位耗时来自结构化工具、检索、重排还是 LLM 调用。

Day 7: Cache Demo + Load Test + Week2 包装

当天目标	实现 mock cache 或 Redis cache demo，加入最小压测脚本，补齐 README、performance.md、week2_summary.md 和简历 bullet 初版。
生产区边界	当天产出的接口、schema、router、tool、evaluation 或 tracer 都会被 Week 3/Week 4 继续依赖；Codex 只能生成候选稿，必须通过人工 Review Gate 后再合并。
不做事项	不一次性重写全项目；不把电商规则写进 core pipeline；不

需要创建或修改的文件

- pipeline/cache.py: 实现 cache interface, 先支持 in-memory/mock cache; Redis 可选。
- scripts/load_test.py 或 locustfile.py: 最小压测 /health、/chat mock LLM。
- docs/performance.md: 写明环境、命令、指标结果、局限。
- docs/week2_summary.md: 总结本周能力、失败案例、未完成项。
- README.md: 更新 Week2 Progress、Quick Start、API 示例、Evaluation、Performance。
- resume_bullets.md 或 README section: 3-5 条 RAG 项目简历 bullet 初版。
- tests/test_cache.py。

给 Codex 的任务卡

Context:

Week2 最后一天要把 RAG 工程化成果收尾, 不继续无限加功能。重点是 cache demo、load test 和文档包装。

Goal:

实现最小 cache demo, 完成 load_test.py, 生成 docs/performance.md 和 docs/week2_summary.md, 并更新 README 与简历 bullet。

Constraints:

1. cache 可先用 in-memory/mock, 不强制 Redis。
2. 性能结果必须来自本地脚本, 不得虚构。
3. load test 不代表生产压测, 只能写为 local mock benchmark。
4. 不继续扩散新功能, 不做复杂 Docker Compose。

Interface & Data:

cache.get(key), cache.set(key, value, ttl_seconds), cache.stats()。

load_test.py 至少能请求 /health 和 /chat。

Process:

先实现 cache interface, 再接入高频 FAQ 或最终回答缓存, 然后写 load_test.py 和 performance.md。

Review:

输出本地测试环境、命令、样本规模、指标结果和不能夸大的结论。

建议实现步骤

36. 实现 cache.py: InMemoryCache, 支持 TTL、hit/miss 统计。
37. 选择缓存对象: 优先缓存高频 FAQ / policy 查询的最终回答或检索结果; 不要默认缓存订单状态。
38. 将 cache_hit 写入 TraceInfo。
39. 实现 scripts/load_test.py: 可以用 requests + time + concurrent.futures, 不强制 Locust。
40. 运行评估和压测脚本, 生成本地报告。
41. 补 docs/performance.md: 环境、接口、命令、平均延迟、P95/P99、error rate、cache hit/miss、局限。
42. 更新 README 和 resume bullets: 所有数字必须能在 performance.md 或 eval_report 中找到。

验收命令 / 手动验收

```
pytest tests/test_cache.py
python -m evaluation.run_eval --cases evaluation/ecommerce_cases.json --out evaluation/eval_report.json
python scripts/load_test.py --endpoint http://localhost:8000/health --requests 100 --concurrency 5
python scripts/load_test.py --endpoint http://localhost:8000/chat --requests 50 --concurrency 3 --mock-llm
```

人工检查 docs/performance.md 中的数字是否都能追溯到脚本输出。

人工检查 README 是否能让别人本地启动并复现实验。

人工 Review Gate

- 不要缓存用户特定订单状态，除非明确 key 包含 user/order 且说明一致性风险。
- load_test.py 要记录 error rate，不只记录成功延迟。
- performance.md 必须写明硬件/本地环境、mock LLM 与真实 LLM 差异。
- 简历 bullet 不能说“生产级”“大规模”，只能说“本地评估/压测流程”。

建议 commit

- feat: add cache demo and cache trace
- bench: add local load test script
- docs: finalize week2 rag performance report

失败记录提示

- 记录 cache 误命中或缓存不该缓存的业务数据。
- 记录压测中出现的错误率和最慢接口。

今日三句面试表达

- 我实现了 cache demo 来验证高频 FAQ 的延迟优化思路，但不会把本地 mock 压测夸大成生产指标。
- 我在 performance.md 中写明测试环境、命令和局限，让简历中的性能数字可复现。
- Week2 结束后，RAG 项目已经具备 SQL/结构化工具、文档检索、证据、fallback、evaluation 和性能追踪的工程化雏形。

5. 本周最终验收清单

类别	验收项
工程升级	Week1 naive RAG v0 已审阅；docs/week2_upgrade_plan.md 完成；README 更新 Week2 能力。
结构化工具	orders/products/refunds 三类数据或 schema 存在；order tool、product tool、refund tool 可用。
意图路由	intent router 支持 order_status、logistics、refund、return_policy、warranty、hybrid、unknown。
Metadata	policy docs 扩充；chunk 带 metadata；retriever 支持 metadata_filter 或预留接口。
Evidence	/chat 返回 answer + evidence；evidence 能表示 structured 与 document 两类来源。
Fallback	低分、缺订单号、未知意图、高风险/证据不足问题可以 fallback，且 fallback_reason 明确。
Evaluation	evaluation/ecommerce_cases.json 至少 30 条；run_eval.py 可运行；docs/evaluation.md 完成。
Performance	performance_tracer 记录关键阶段耗时；trace_id 可追踪；docs/performance.md 有可复现实验。
Cache/压测	cache demo 或 mock cache 可用；load_test.py/locustfile.py 可测 /health 和 /chat mock。
失败资产	docs/failure_cases.md 至少记录检索失败、缺订单号、低置信度、政策冲突/高风险问题。
代码留痕	本周至少 8-10 条 RAG 相关 commit，每条对应一个可 review 小功能。
面试资产	形成 3-5 条简历 bullet 初版，所有数字能在 eval/performance 文档中找到来源。

6. 附录：模板

6.1 每天发给 Codex 的六段式 Prompt 模板

Context:

当前项目是 enterprise-rag-copilot，处于 Week2 RAG 工程化阶段。当前模块位置、已有文件、相关接口如下：...

Goal:

本次只完成一个可审查的小功能：...

Constraints:

1. AI 输出只是候选稿。
2. 不把电商业务规则写进 pipeline/。
3. 不新增不必要依赖。
4. 不修改无关文件。
5. 不虚构评估或性能指标。

Interface & Data:

输入结构：...

输出结构：...

错误语义：...

边界样例：...

Process:

请先给设计方案和修改文件清单，再给代码。代码完成后给测试点和验收命令。不要一次性大范围重写。

Review:

请按 Correctness / Readability / Maintainability / Evolvability 给出自查，并标记 Must Fix / Should Improve / Follow-up。

6.2 PR Review Checklist

Correctness:

- ☐ 是否真正解决当天目标？
- ☐ 是否覆盖成功路径和失败路径？
- ☐ 结构化问题是否走 structured tools？
- ☐ 文档问题是否走 retriever + metadata filtering？
- ☐ answer 是否基于 evidence？

Readability:

- ☐ 命名是否能表达意图？
- ☐ 函数是否职责单一？
- ☐ 控制流是否容易追踪？

Maintainability:

- ☐ 电商规则是否只在 domains/ecommerce/？
- ☐ evaluation 是否集中在 evaluation/？
- ☐ performance/tracing 是否没有污染业务逻辑？

Evolvability:

- ☐ 换成 HR/IT domain 时，core pipeline 是否可复用？
- ☐ 后续接真实 DB / Redis / reranker 时是否有接口空间？

Decision:
Must Fix:
Should Improve:
Follow-up:
Accept / Reject:

6.3 失败记录模板

Date:
Case ID:
Input Query:
Expected Intent / Route:
Actual Intent / Route:
Expected Evidence:
Actual Evidence:
Fallback Triggered: yes/no
Fallback Reason:
Trace ID:
Root Cause:
[] intent error
[] structured tool error
[] retrieval miss
[] metadata filter too strict
[] evidence missing
[] hallucinated answer
[] latency/cache issue
Fix Plan:
Regression Test Needed:

6.4 Week2 简历 Bullet 初版模板

- 设计并实现可迁移企业级 RAG Copilot 的 Hybrid RAG 流程，将订单/退款/商品等精确事实查询抽象为 structured tools，将退换货/物流/保修等政策问答交给文档检索。
- 实现 intent router、metadata-aware retriever、evidence builder 和 fallback handler，使系统支持多路由问答、证据引用和低置信度拒答/转人工。
- 构建 30+ 条 ecommerce evaluation cases，覆盖结构化问题、文档问题、混合问题和失败问题，并实现基础 evaluation runner 统计 intent、retrieval、fallback 与 evidence 相关指标。
- 设计 performance tracer 与本地压测脚本，记录 tool/retrieval/rerank/LLM/total latency、error rate 和 cache hit/miss，所有性能数字均来自本地可复现实验。

7. Week2 最重要的一句话

Week2 不是为了把 RAG 项目写得“更复杂”，而是为了让它从“能回答的 naive demo”升级为“可解释、可评估、可追踪、失败可控、指标可复现”的工程化 RAG 雏形。